Hugo Mellado Ibáñez

Activity 1.2. A network file system

The evolution toward modern cloud computing (where hardware and network complexities are entirely abstracted to provide a single system image) is built upon the foundation of early networked applications. The SAR/SZA network file service serves as a practical baseline to understand this progression. As a foundational client-server architecture relying on explicit message passing, it introduces basic resource distribution, concurrency, and remote access capabilities. This report evaluates the SAR/SZA file service by identifying which core properties of distributed systems it successfully implements. Additionally, it highlights the system's structural limitations by contrasting its centralized, explicit architecture with the advanced transparency, scalability, and dependability required of an ideal cloud-based distributed file system.

Properties Met by the SAR/SZA Network File Service

- **Client-Server Architecture:** The service operates as a networked system utilizing a basic client-server model over standard TCP/IP INET sockets.
- **Concurrency:** The server is capable of handling multiple simultaneous client connections by employing the os.fork() system call to create parallel processes for each active dialog.
- **Access Control and State Management:** The system implements foundational security through identification and authentication states, verifying specific users and passwords. It also enforces basic permissions, explicitly preventing anonymous users from uploading or deleting files.
- **Message Passing Communication:** The service relies on explicit message passing, utilizing a custom ASCII-based protocol with predefined commands (e.g., List, Download, Upload, Delete) to coordinate actions between the distributed nodes.

Properties Lacking in Relation to a Cloud System (Ideal Distributed File System)

- **Transparency:** The service fundamentally lacks access, location, and migration transparency. Instead of hiding where a resource is located or how it is accessed, users must explicitly define the server address, connect via a specialized client menu, and manually push or pull files across the network.
- **Single System Image (SSI):** Unlike an ideal distributed system, this application does not appear to its users as a single coherent system. The boundary between the local file system and the remote server's FILES_PATH directory is entirely visible to the user.
- **Scalability:** The architecture is strictly centralized around a single server binding to a specific port and a single local storage path, lacking the modularity to grow capacity seamlessly. It does not implement distributed hierarchical namespaces, caching, or mirroring to maintain efficiency under heavy loads.

- **Dependability and Fault-Tolerance:** The service represents a single point of failure without active or passive replication mechanisms. If the server hardware fails or the process crashes, file availability is lost entirely, meaning it cannot maintain correct functioning despite component failure.
- **Consistency Management:** Because the system does not replicate data across multiple nodes, it bypasses the complexities of managing global state or concurrent replication. However, it lacks advanced consistency mechanisms like transactions (ACID properties) that a cloud system would require to safely manage concurrent read/write operations on the same distributed resource.

In conclusion, the SAR/SZA network file service provides a clear, functional example of early networked applications operating within a Network Operating System paradigm. By implementing socket-based communication, explicit protocol command exchanges, user authentication, and process-level concurrency, it successfully enables basic remote file sharing and management across independent machines.

However, its reliance on direct server addressing, manual file operations, and a single centralized storage architecture reveals significant shortcomings compared to modern cloud environments. It fundamentally lacks critical distributed properties, including location and access transparency, fault tolerance, horizontal scalability, and automated consistency mechanisms. Ultimately, analysing this service highlights the engineering progression from simple networked systems to cloud file services, underscoring why middleware layers are essential to delivering a truly transparent, scalable, and dependable Single System Image.